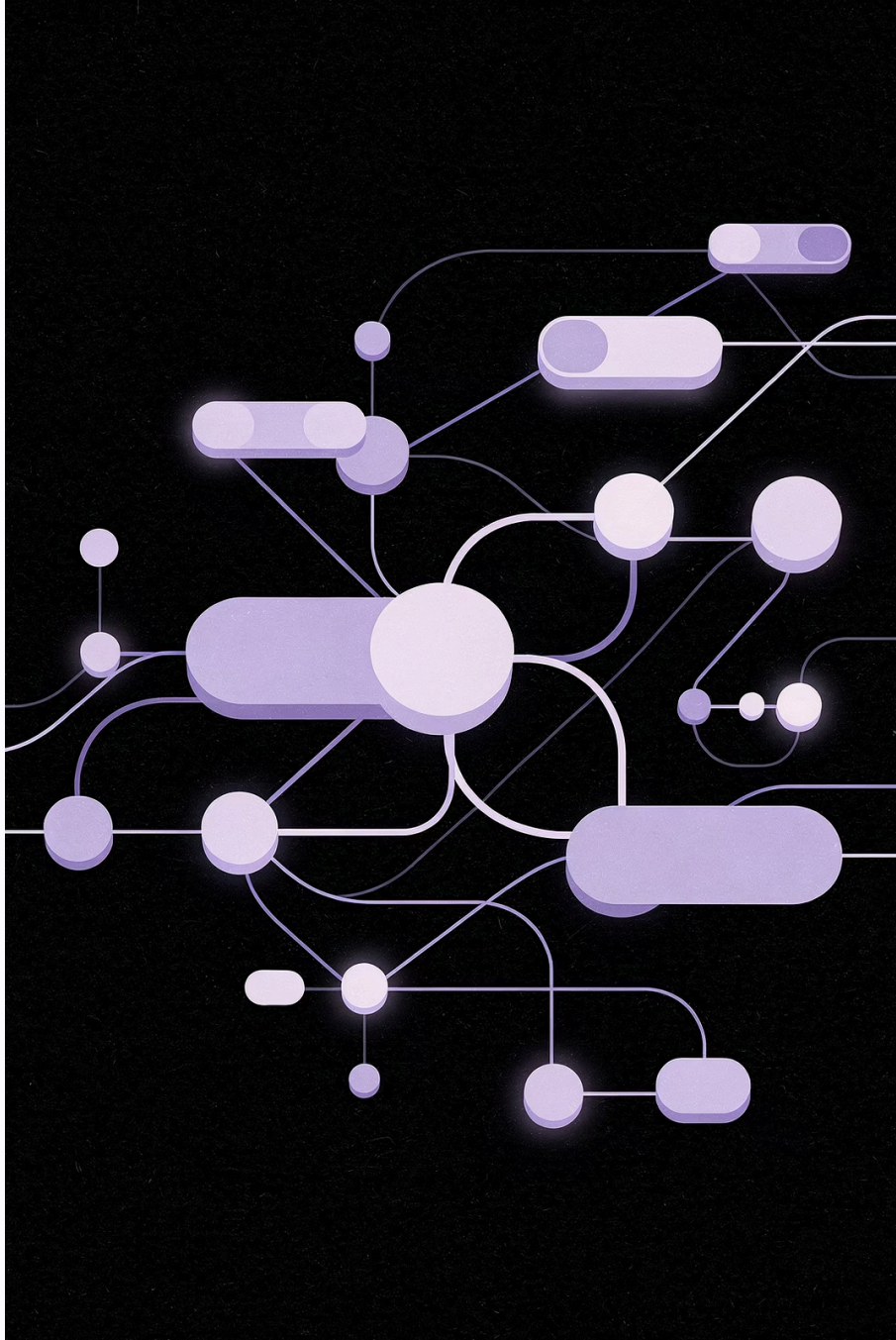
Core Concepts in Databricks

A practical introduction to the foundational building blocks of the Databricks platform — designed for data engineers and data scientists ready to hit the ground running.

GETTING STARTED

DATABRICKS FUNDAMENTALS



The Databricks Workspace

The **Workspace** is your central hub for all activity within Databricks. Think of it as the control panel from which you manage every asset — notebooks, clusters, jobs, and data — in one unified interface.

Navigation Sidebar

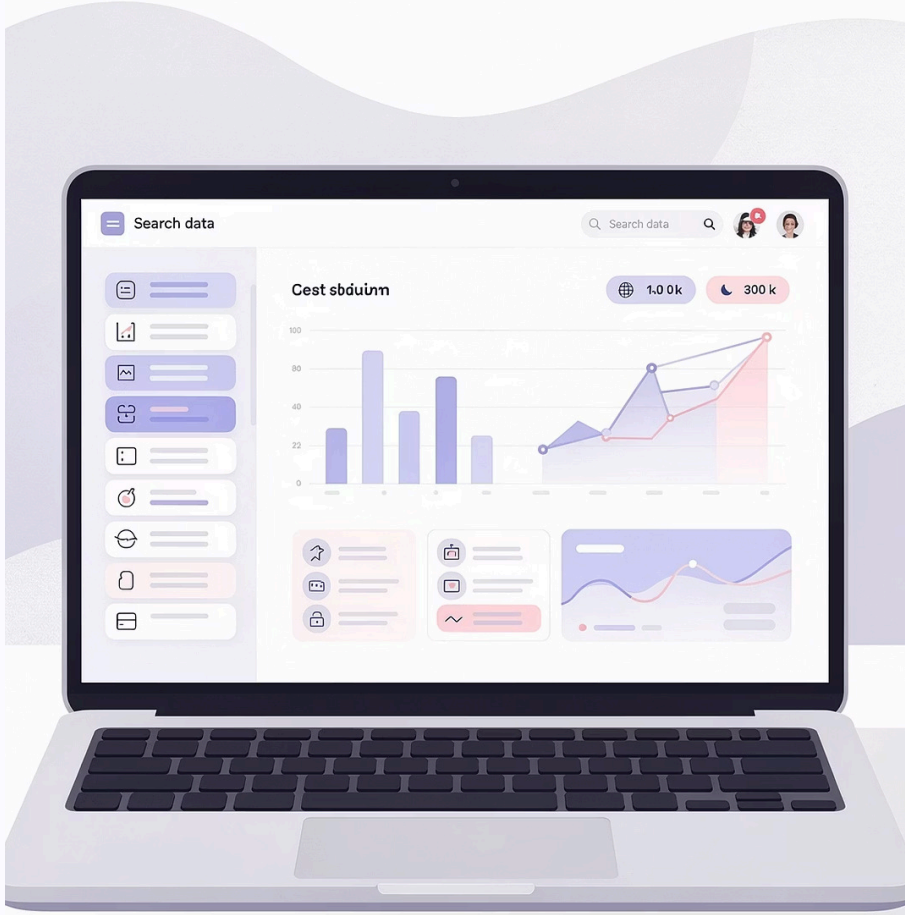
Quickly access notebooks, clusters, jobs, the data catalogue, and settings from the persistent left-hand menu.

Folders & Permissions

Organise assets in a hierarchical folder structure with fine-grained access controls for individuals and teams.

Collaboration Ready

Share notebooks and resources with colleagues directly within the platform, enabling real-time collaborative development.



Notebooks

Databricks **Notebooks** are the primary interactive environment for writing and executing code. They support multiple languages within a single document, making them remarkably flexible for mixed workflows.

Multi-Language Support

Switch between **Python**, **SQL**, **Scala**, and **R** using magic commands such as `%python` or `%sql` at the top of any cell.

Interactive Execution

Run individual cells and inspect outputs immediately — tables, charts, and logs appear inline, accelerating the iterative development cycle.

Version History

Built-in revision history lets you review, compare, and restore previous versions of your notebook without any external tooling.

Supported Languages

- **Python** — PySpark, pandas, ML libraries
- **SQL** — Direct table queries and transformations
- **Scala** — Native Spark API access
- **R** — Statistical analysis and visualisation

Key Shortcuts

- `%run` — Execute another notebook
- `%fs` — File system commands
- `%sh` — Shell commands on the driver

Clusters

A **cluster** is the distributed compute engine that powers your notebooks and jobs. Databricks manages the underlying infrastructure, so you focus on code rather than configuration.

All-Purpose Clusters

Shared, always-on compute environments ideal for **interactive development**, data exploration, and ad-hoc analysis. Multiple users can attach notebooks simultaneously, making them perfect for collaborative work sessions.

Job Clusters

Ephemeral clusters that spin up automatically when a **scheduled job** starts and terminate once it completes. They are cost-efficient because you pay only for the exact duration of the workload — no idle time.

Cluster Configuration

Choosing the right cluster configuration directly impacts both **performance** and **cost**. Databricks provides powerful options to match compute resources to workload demands dynamically.

Autoscaling

Databricks automatically adds or removes worker nodes based on current workload. Set a **minimum and maximum** worker count and let the platform optimise resource usage — preventing both under-provisioning and unnecessary spend.

Instance Types

Select from cloud provider instance families — **memory-optimised** for large joins and caches, **compute-optimised** for CPU-intensive transformations, and **GPU instances** for deep learning workloads.

Auto-Termination

Configure clusters to shut down automatically after a defined period of inactivity — a simple but effective guard against runaway costs from forgotten interactive sessions.

Runtime & Libraries

Choose a **Databricks Runtime** version that bundles Spark, Delta Lake, and popular ML libraries. Install additional libraries via PyPI, Maven, or uploaded JARs directly on the cluster.

DBFS — Databricks File System

The **Databricks File System (DBFS)** is an abstraction layer that presents cloud object storage as a familiar, POSIX-like file system. You interact with paths such as `dbfs:/` without needing to manage raw cloud storage APIs.

Abstraction Layer

DBFS translates standard file-system operations — read, write, list, delete — into the corresponding calls against the underlying cloud storage, shielding you from cloud-specific complexity.

Backed by Cloud Object Storage

By default, DBFS is backed by **Amazon S3**, **Azure Data Lake Storage**, or **Google Cloud Storage** depending on your deployment, providing virtually unlimited, durable storage at low cost.

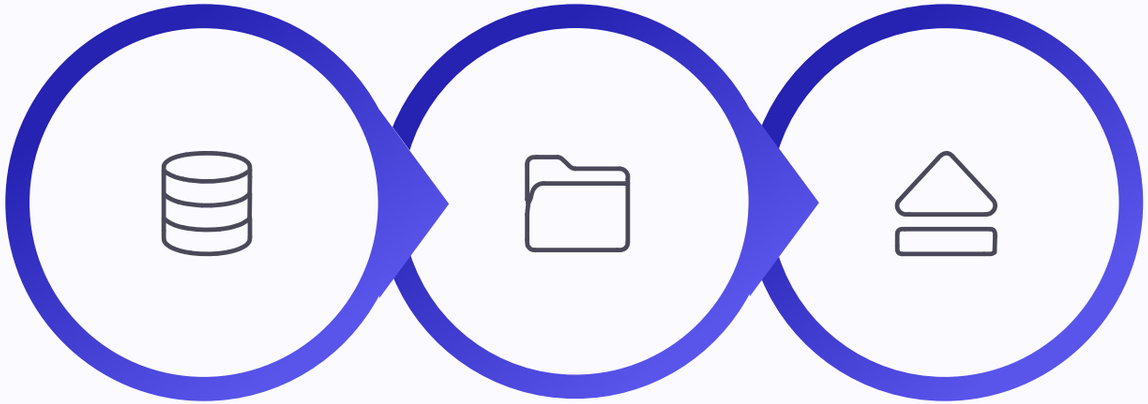
Common DBFS Paths

- `dbfs:/FileStore/` — User-uploaded files and charts
- `dbfs:/databricks-datasets/` — Curated sample datasets
- `dbfs:/user/hive/warehouse/` — Managed table storage
- `dbfs:/tmp/` — Temporary working files

Access DBFS from a notebook using `%fs ls /` or the `dbutils.fs` API in Python and Scala.

Mount Points

Mount points bridge the gap between external cloud storage and the Databricks environment. By mounting a storage location, you can access it using a clean, readable path instead of long cloud URIs.



Create Mount

Access Data

Unmount

Why Use Mount Points?

- Simplify paths — use `/mnt/raw-data/` instead of full S3 or ADLS URIs
- Centralise credentials — authenticate once at mount time
- Collaborate easily — all users share the same logical path

Example Usage

```
dbutils.fs.mount(
 source="s3a://my-bucket/raw",
 mount_point="/mnt/raw-data",
 extra_configs={...}
)
df = spark.read.parquet("/mnt/raw-data/")
```

Jobs & Workflows

Databricks **Jobs** transform notebooks and scripts into reliable, automated pipelines. The Workflows engine handles scheduling, dependency resolution, error handling, and retries — so production pipelines run without manual intervention.

1

Define Tasks

Compose a workflow from notebooks, Python scripts, JAR files, or dbt projects — mix and match task types in a single pipeline.

2

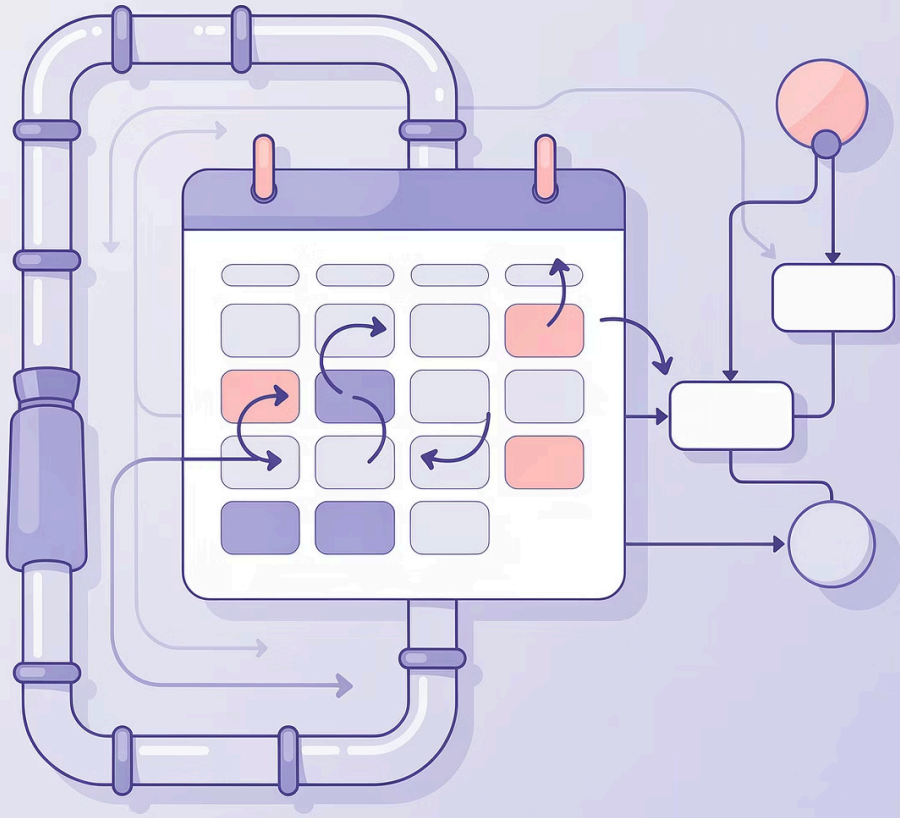
Set Schedule

Trigger on a cron schedule, on arrival of new data, or via the **REST API** for event-driven execution from external systems.

3

Monitor & Alert

Track run status, duration, and logs in the Workflows UI. Configure **email or webhook alerts** on failure or success.



Delta Lake — A Preview

What is Delta Lake?

Delta Lake is an **open-source storage layer** built on top of Parquet files that brings database-grade reliability to your data lake. It is the default table format in Databricks and underpins the entire Lakehouse architecture.

Why it Matters

Traditional data lakes suffer from inconsistency, failed writes, and lack of versioning. Delta Lake solves these problems with a transaction log that records every change to your data.

1

ACID Transactions

Guarantees **Atomicity, Consistency, Isolation, and Durability**. Concurrent reads and writes are safe — partial failures never leave your table in a corrupt state.

2

Time Travel

Query any previous version of a table using `VERSION AS OF` or `TIMESTAMP AS OF`. Audit changes, reproduce experiments, or roll back accidental deletes instantly.

3

Schema Enforcement

Delta Lake rejects writes that don't conform to the table schema by default, preventing data quality issues from propagating downstream.

Now Let's Build Your First Pipeline

You've covered the core concepts — Workspace, Notebooks, Clusters, DBFS, Mount Points, Jobs, and Delta Lake. Now it's time to put that knowledge into practice.

01

Attach to a Cluster

Open your assigned notebook and connect it to the shared all-purpose cluster.

02

Read Data from DBFS

Load the sample dataset using `spark.read` and inspect the schema.

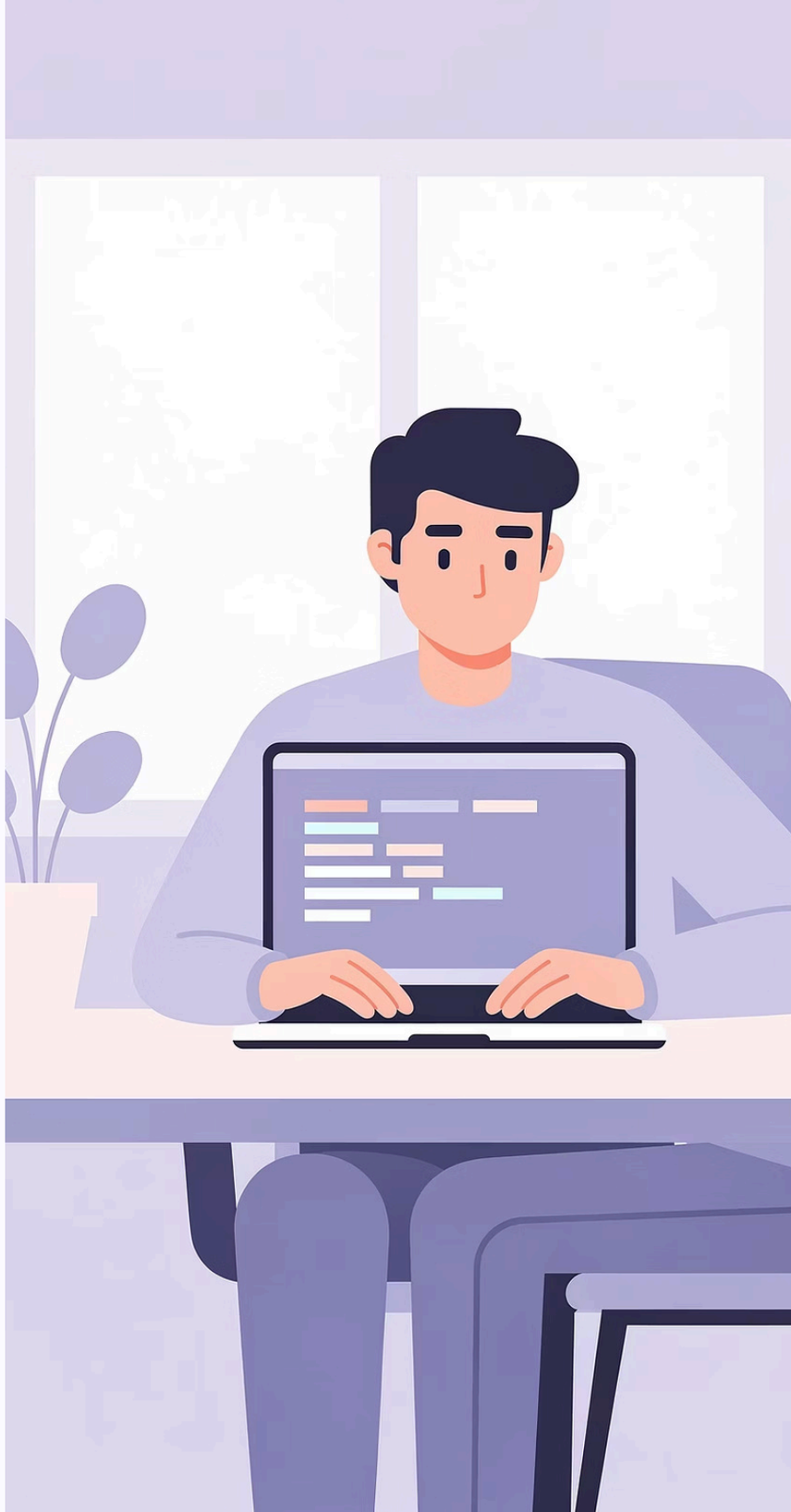
03

Transform with PySpark or SQL

Apply filters, aggregations, and joins to shape the data into your target structure.

04

Write to a Delta Table

Persist your results as a managed Delta table and query it with time travel.

> **Tip:** If you get stuck, use `dbutils.help()` or raise your hand — the lab guide covers every step in detail.